

Kapitel 4: Algorithmische Grundstrukturen und ihre Darstellung

Bisher haben wir hauptsächlich Programme gesehen, bei denen der Programmcode Zeile für Zeile nacheinander ausgewertet wird. Mit Hilfe **algorithmischer Grundstrukturen** können wir diesem Verhalten abweichen.

Algorithmische Grundstrukturen steuern dabei den Ablauf eines Programms. Mit ihnen könnt ihr Anweisungsblöcke definieren, die nur unter einer bestimmten Bedingung ausgeführt werden oder auch solche, deren Ausführung mehrfach erfolgt. Hierfür unterscheiden wir **Sequenzen**, **Verzweigungen** und **Schleifen**, welche wir uns - inklusiv dreier **Darstellungsformen** - nachfolgend genauer anschauen.

4.8. Schleifen / Iteration

Betrachten wir zunächst das nachfolgende **Code-Beispiel**.

```
# FÜHRE MICH UNBEDINGT AUS!
```

```
print(1)
print(2)
print(3)
print(4)
print(5)
print(6)
print(7)
print(8)
print(9)
print(10)
print(11)
print(12)
```

Was passiert hier?

Zwölfmal wird immer und immer wieder die gleiche Funktion aufgerufen. Der Code ist nahezu identisch. Lediglich die ausgegebene Zahl ändert sich.

Jetzt stellen wir uns einmal vor, dass wir die Zahlen von 1 bis 100 ausgeben wollen.

100 mal den gleichen Code schreiben? Klingt nicht so spaßig? Stimmt! Dafür gibt es aber eine algorithmische Lösung: **die Schleife!**

Die Schleife als algorithmische Grundstruktur ermöglicht es uns, Code **mehrfach hintereinander auszuführen**, ohne immer und immer wieder die gleichen Code-Zeilen schreiben zu müssen.

Das wiederholte Ausführen von Anweisungen wird bei der Programmierung als **Iteration** bezeichnet.

Eine **Iteration** ist ein einzelner Durchlauf innerhalb einer Schleife. Wenn also eine Schleife viermal ausgeführt wird, dann besteht sie aus vier Iterationen.

Wir unterscheiden grundsätzlich **3 Arten von Schleifen**:

- Kopfgesteuerte Schleife
- Fußgesteuerte Schleife
- Zählschleife

Diese werden nachfolgend näher betrachtet.

4.8.1. Kopfgesteuerte Schleife

Bei der **kopfgesteuerten Schleife** wird ein Code-Block so lange wiederholt ausgeführt, bis eine Bedingung (im Kopf der Schleife) **nicht mehr erfüllt** ist.

Im **Pseudocode** könnte man exemplarisch folgendes formulieren:

```
SOLANGE zaehler < 5  
    zaehler = zaehler + 1
```

Gut zu sehen ist, dass die letzten zwei Zeilen eingerückt sind. Das verdeutlicht, dass alles, was eingerückt ist, von der Schleife wiederholt wird.

Nachfolgend ist dieses Beispiel als **Code und Struktogramm** visualisiert.

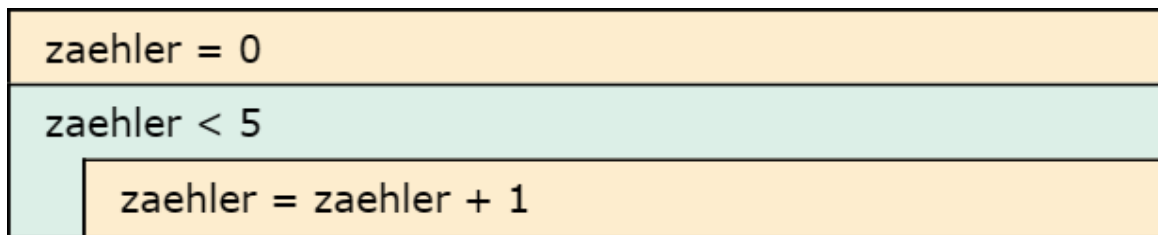
Zu sehen ist bei den Beispielen auch, dass vor der Schleife die Variable `zaehler` mit dem Wert `0` initialisiert wurde.

In **JEDEM** Schleifendurchlauf wird der `zaehler` um eins **inkrementiert** (das Fachwort für: 'erhöht'), solange bis die Bedingung im Schleifenkopf **nicht mehr erfüllt** ist.

Programmcode:

```
zaehler = 0  
  
while zaehler < 5:  
    zaehler = zaehler + 1
```

Struktogramm:



In die geometrische Form für die Schleife (grüne Form) wird lediglich die Bedingung eingetragen.

Dass es sich dabei um eine Schleife handelt, kann aus der Form abgeleitet werden.

Programmcode und Struktogramm mit Ausgaben zur besseren Nachvollziehbarkeit:

```
# FÜHRE MICH UNBEDINGT AUS!
```

```
zaehler = 0  
print(f'Zähler VOR der Schleife: {zaehler}')
```

```
while zaehler < 5:  
    zaehler = zaehler + 1
```

```
print(f'Zähler NACH der Schleife: {zaehler}')
```

zaehler = 0

A: 'Zähler VOR der Schleife: ' + zaehler

zaehler < 5

zaehler = zaehler + 1

A: 'Zähler NACH der Schleife: ' + zaehler

Programmcode und Struktogramm mit erweiterten Ausgaben zur besseren Nachvollziehbarkeit:

```
# FÜHRE MICH UNBEDINGT AUS!
```

```
zaehler = 0
print(f'Zähler VOR der Schleife: {zaehler}')

print('- - -')

while zaehler < 5:
    print(f'Berechnet wird nun der neue Wert des Zählers mit: {zaehler} + 1.')
    zaehler = zaehler + 1
    print(f'Der neue Wert von Zähler ist {zaehler}.')

print('- - -')

print(f'Zähler NACH der Schleife: {zaehler}')
```

zaehler = 0
A: 'Zähler VOR der Schleife: ' + zaehler
A: '- - -'
zaehler < 5
A: 'Berechnet wird nun der neue Wert des Zählers mit: ' + zaehler + ' + 1.'
zaehler = zaehler + 1
A: 'Der neue Wert von Zähler ist ' + zaehler + '.'
A: '- - -'
A: 'Zähler NACH der Schleife: ' + zaehler

Hinweise zum Code einer kopfgesteuerten Schleife:

- Jede kopfgesteuerte Schleife beginnt mit dem Schlüsselwort `while`.
- Danach folgt eine Bedingung, die wahr oder falsch sein kann. In PYTHON bedeutet dies, dass sie als Ergebnis den Wert `True` oder `False` haben muss. Nur im Falle von True werden die nun folgenden eingerückten Anweisungen auch abgearbeitet.
- Der Doppelpunkt `:` schließt die Bedingung ab.
- Alle Anweisungen, die bei Erfülltsein der Bedingung abgearbeitet werden sollen, müssen gleich weit eingerückt sein. Dies können auch mehrere Anweisungen sein. Nutze Tabulatoren (die Taste mit den Doppelpfeilen am linken Rand der Tastatur) zum Einrücken. Man bezeichnet diese Anweisungen als Schleifenkörper.
- Eine Schleife, d. h. der Schleifenkörper, darf niemals leer sein. Es muss mindestens eine eingerückte Anweisung vorhanden sein. Kommentarzeilen zählen dabei natürlich nicht.
- Die erste nicht eingerückte Anweisung kennzeichnet, dass die Schleife zu Ende ist.
- Damit die Schleife nicht zu einer Endlosschleife wird, muss sichergestellt werden, dass die Bedingung irgendwann im Laufe des Programmlaufs nicht mehr erfüllt ist.

Beispiel 1:

Schauen wir uns das Ganze zum besseren Verständnis nochmal etwas genauer an.

Wir betrachten dazu den folgenden Code.

```
# FÜHRE MICH UNBEDINGT AUS!

i = 0

while i < 10:
    i = i + 1
```

i = 0

i < 10

i = i + 1

Was passiert hier eigentlich? Das zeigt für (fast) jeden Schritt die nachfolgende Tabelle.

Schritt	Bedingung i < 10?	Aktion	neuer Wert von i
0	-	Initialisierung (vor der eigentlichen Schleife)	i = 0
1	✓ WAHR (0 < 10)	i = 0 + 1 → 1	i = 1
2	✓ WAHR (1 < 10)	i = 1 + 1 → 2	i = 2
3	✓ WAHR (2 < 10)	i = 2 + 1 → 3	i = 3
...	...	...	...
9	✓ WAHR (8 < 10)	i = 8 + 1 → 9	i = 9
10	✓ WAHR (9 < 10)	i = 9 + 1 → 10	i = 10
	✗ FALSCH (10 < 10)	Schleife wird verlassen	i = 10 (Endwert)

Die Bedingung `i < 10` wird vor jedem Durchlauf geprüft. Bei `i = 10` ist `10 < 10` falsch und die Schleife bricht ab.

Beispiel 2:

Ein ähnliches Beispiel kann man mit Zeichenketten konstruieren.

Wir behalten unser `i` bei und lassen die Zeichenkette in jedem Schleifendurchlauf wachsen, indem die Zahl als Zeichen hinzugefügt wird.

```
# FÜHRE MICH UNBEDINGT AUS!
```

```
i = 0
text = ''

while i < 10:
    text = text + str(i)
    print(f'i = {i} --> text = {text}')
    i = i + 1
```

`i = 0`

`text = ''`

`i < 10`

`text = text + i`

A: `'i = ' + i + ' --> text = ' + text`

`i = i + 1`

LearningView.org



4.8.2. Übungen zur kopfgesteuerten Schleife

Aufgabe 1

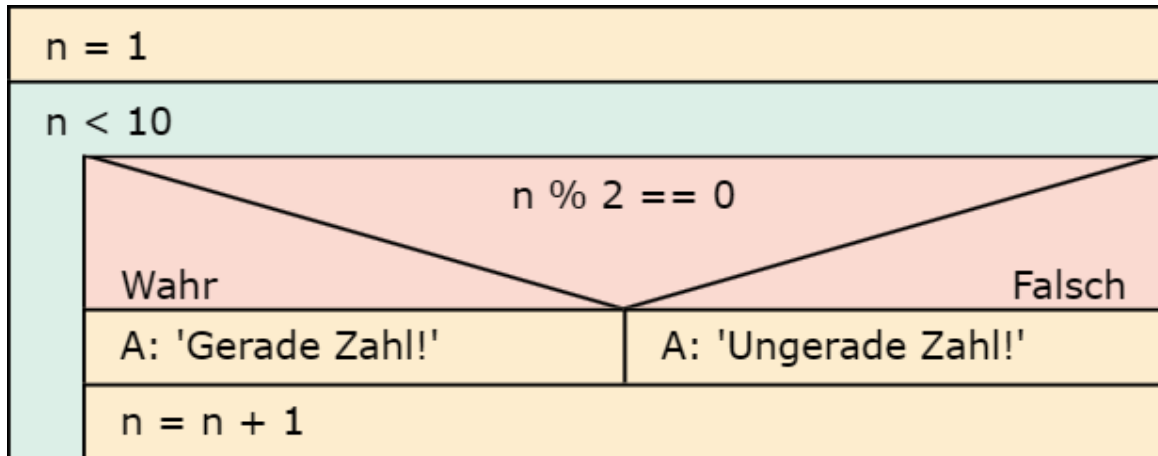
Implementiere den folgenden Pseudocode.

```
Eingabe einer Zahl n  
Setze die Variable Zähler auf 1  
SOLANGE der Zähler kleiner n ist  
    Erhöhe Zähler um 0.125  
Ausgabe des Zählers
```

```
# HIER IST PLATZ FÜR DEINE LÖSUNG! :)
```

Aufgabe 2

Implementiere das folgende Struktogramm.

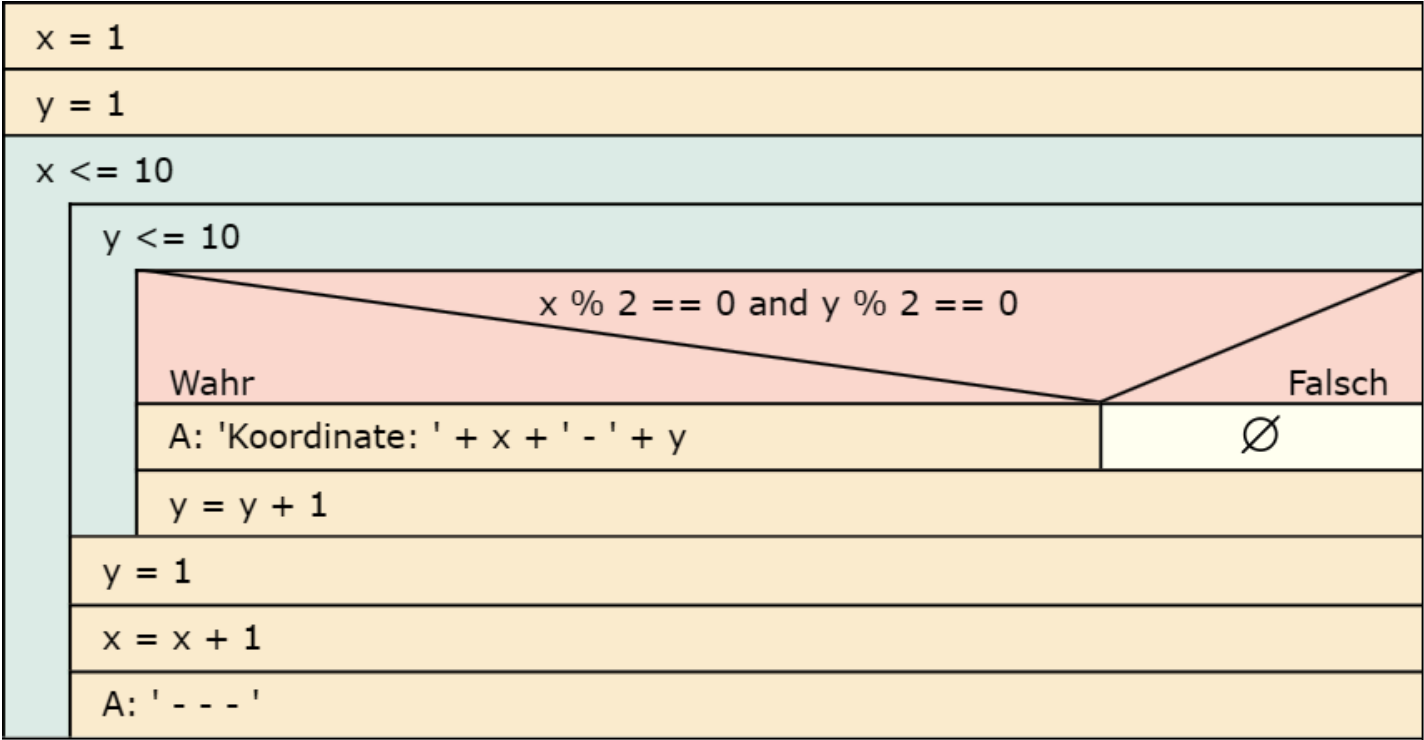


Besonders zu beachten ist, dass die letzte Anweisung `n = n + 1` über die gesamte Breite geht. Das bedeutet, dass diese Anweisung **NICHT MEHR** zur Verzweigung gehört!

HIER IST PLATZ FÜR DEINE LÖSUNG! :)

Aufgabe 3

Implementiere das folgende Struktogramm.



Hinweis: Damit die Ausgabe innerhalb der Verzweigung fehlerfrei funktioniert muss der Datentyp der Variablen `x` und `y` entsprechend konvertiert werden.

HIER IST PLATZ FÜR DEINE LÖSUNG! :)

Aufgabe 4

Implementiere ein Programm, welches nach Eingabe einer oberen Grenze, alle Zahlen von 0 bis n aufaddiert.

Bei der Eingabe der Zahl 3 ergibt sich beispielsweise das Ergebnis 6.

Gerechnet wird dabei: $0 + 1 + 2 + 3 = 6$.

```
# HIER IST PLATZ FÜR DEINE LÖSUNG! :)
```

